

AIML SESSION 10 ASSIGNMENT SUBMISSION

Name of Student : Vaibhav Vijay Kate

Domain/Subject : AIML (Artificial Intelligence & Machine Learning)

College : Government Polytechnic College Washim

Branch : Information Technology

Session : 10

Q1

Q1. Basic Class and Object)

Create a class named Car with:

Attributes: brand and model (set using __init__).

A method display_info() that prints the brand and model.

Create two objects of the Car class with different values and call the method for both.

Program Code:

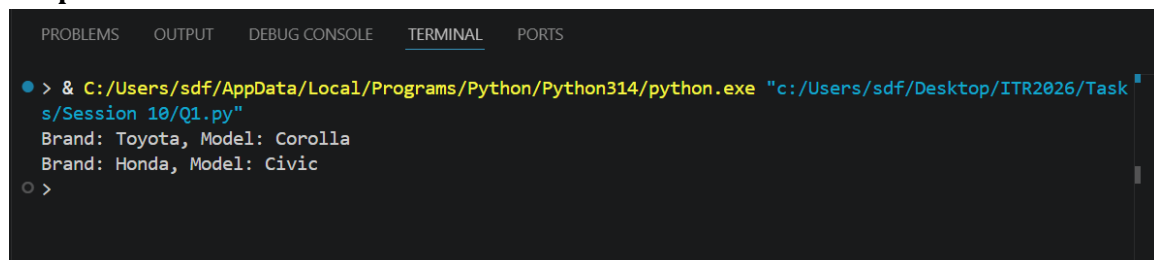
```
# Create a class named Car with:
class Car:
    # Attributes: brand and model (set using __init__).
    def __init__(self, Brand, Model):
        self.Brand = Brand
        self.Model = Model

# A method display_info() that prints the brand and model.
    def display_info(self):
        print(f"Brand: {self.Brand}, Model: {self.Model}")

# Initialize two Car objects and call the method for both
car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")

car1.display_info()
car2.display_info()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Task
s/Session 10/Q1.py"
Brand: Toyota, Model: Corolla
Brand: Honda, Model: Civic
>
```

Q2

Q2. Using init Constructor)

Write a class Book that takes title, author, and price as parameters in `__init__`.

Create a method `show_details()` to print all the book information.

Create at least two book objects and display their details.

Program Code:

```
# Write a class Book that takes title, author, and price as parameters in __init__.
```

```
class Book:
```

```
    def __init__(self,title,author,price):
        self.title=title
        self.author=author
        self.price=price
```

```
# Create a method show_details() to print all the book information.
```

```
    def show_details(self):
        print("\nTitle:",self.title)
        print("Author:",self.author)
        print("price:",self.price)
```

```
# Create at least two book objects and display their details.
```

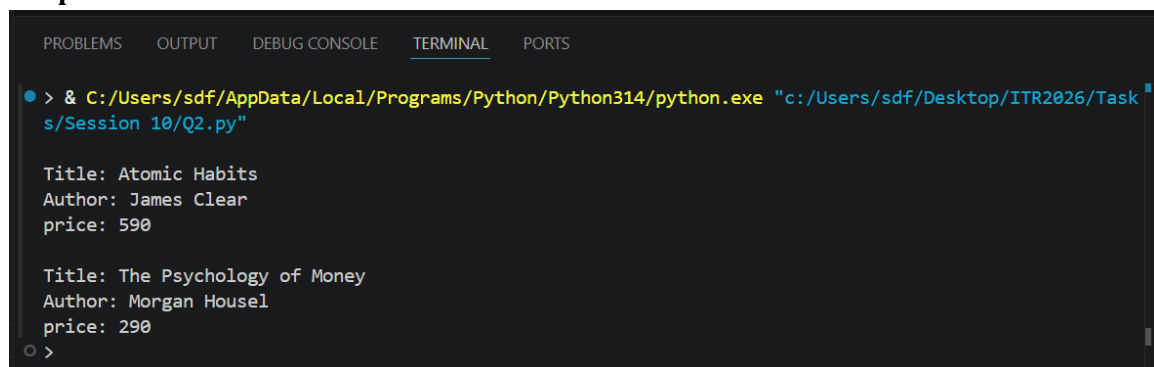
```
b1=Book("Atomic Habits","James Clear","590")
```

```
b2=Book("The Psychology of Money","Morgan Housel","290")
```

```
b1.show_details()
```

```
b2.show_details()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Task
s/Session 10/Q2.py"

Title: Atomic Habits
Author: James Clear
price: 590

Title: The Psychology of Money
Author: Morgan Housel
price: 290

>
```

Q3

Q3. Instance Methods and self)

Create a class BankAccount with:

Attributes: account_holder and balance (initialized in __init__).

Methods:

deposit(amount) — add amount to balance.

withdraw(amount) — subtract amount from balance (only if sufficient balance).

show_balance() — print current balance.

Create an object and demonstrate all methods.

Program Code:

Create a class BankAccount with:

Attributes: account_holder and balance (initialized in __init__).

class BankAccount:

def __init__(self, account_holder, balance):

self.account_holder = account_holder

self.balance = balance

deposit(amount) — add amount to balance.

def deposit(self, amount):

if amount > 0:

self.balance += amount

print(f'{amount} deposited. New balance: {self.balance}')

else:

print("Deposit amount must be positive.")

withdraw(amount) — subtract amount from balance (only if sufficient

def withdraw(self, amount):

if amount <= 0:

print("Withdraw amount must be positive.")

elif amount > self.balance:

print("Insufficient balance.")

else:

self.balance -= amount

print(f'{amount} withdrawn. New balance: {self.balance}')

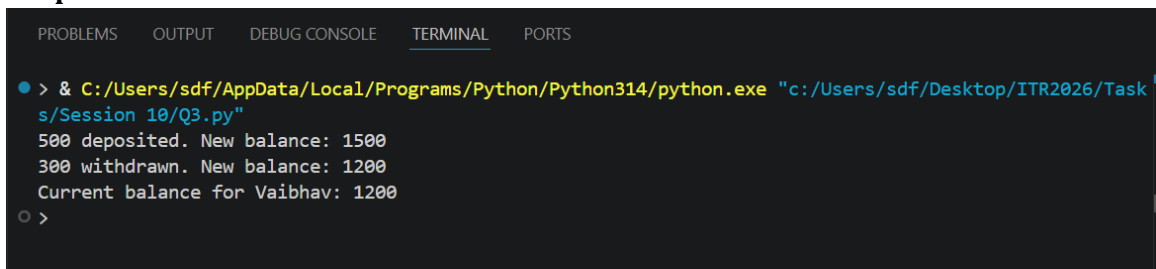
show_balance() — print current balance.

def show_balance(self):

print(f'Current balance for {self.account_holder}: {self.balance}')

```
if __name__ == "__main__":  
    # Create an object and demonstrate all methods.  
    account = BankAccount("Vaibhav", 1000)  
    account.deposit(500)  
    account.withdraw(300)  
    account.show_balance()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q3.py"  
500 deposited. New balance: 1500  
300 withdrawn. New balance: 1200  
Current balance for Vaibhav: 1200  
○ >
```

Q4

Q4. Method with Parameters)

Create a class Student with attributes name and marks.

Add a method calculate_grade() that returns "Pass" if marks $\geq$ 40, otherwise "Fail".

Create 3 student objects, display their details, and print their grades.

Session 10 (AIML) - Assignment Questions 1

Program Code:

```
# Create a class Student with attributes name and marks.
```

```
class Student:
```

```
    # Add a method calculate_grade() that returns "Pass" if marks  $\geq$  40, otherwise Fail
```

```
    def __init__(self,name,marks):
```

```
        self.name =name
```

```
        self.marks =marks
```

```
    def calculate_grade(self):
```

```
        if self.marks >= 40:
```

```
            return "Pass"
```

```
        else:
```

```
            return "Fail"
```

```
# Create 3 student objects, display their details, and print their grades.
```

```
Student1 = Student ("Vaibhav" , 89)
```

```
Student2 = Student ("Ram" , 91)
```

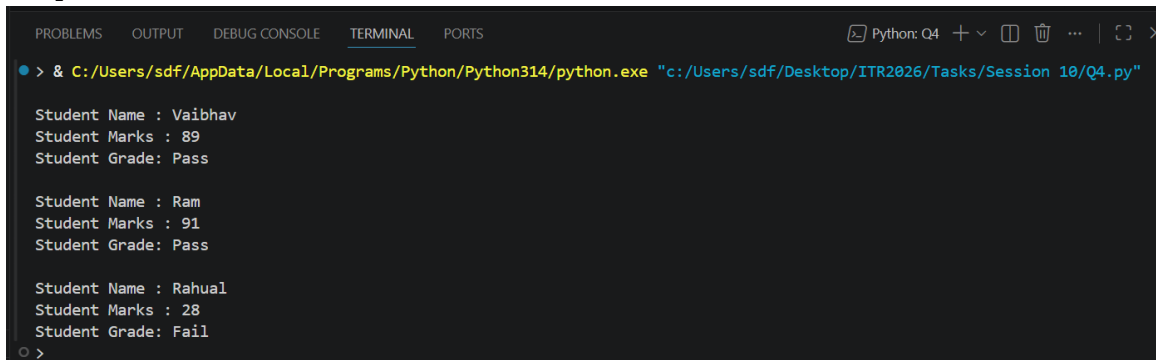
```
Student3 = Student ("Rahual" , 28)
```

```
print("\nStudent Name :", Student1.name,"\nStudent Marks :",Student1.marks,"\nStudent Grade:",Student1.calculate_grade())
```

```
print("\nStudent Name :", Student2.name,"\nStudent Marks :",Student2.marks,"\nStudent Grade:",Student2.calculate_grade())
```

```
print("\nStudent Name :", Student3.name,"\nStudent Marks :",Student3.marks,"\nStudent Grade:",Student3.calculate_grade())
```

Output Screenshot:



The screenshot shows a VS Code terminal window with the following tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The terminal title bar indicates the file is 'Python: Q4'. The command executed is `& C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q4.py"`. The output displays student information for three students: Vaibhav (89 marks, Pass), Ram (91 marks, Pass), and Rahual (28 marks, Fail).

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q4.py"

Student Name : Vaibhav
Student Marks : 89
Student Grade: Pass

Student Name : Ram
Student Marks : 91
Student Grade: Pass

Student Name : Rahual
Student Marks : 28
Student Grade: Fail

>
```

Q5

Q5. Multiple Methods)

Create a class Employee with:

Attributes: name, salary (use __init__).

Methods:

display_details()

give_raise(amount) — increase salary and print new salary.

yearly_bonus() — return 10% of current salary.

Create an employee object and demonstrate all methods.

Program Code:

```
# Create a class Employee
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display_details(self):
        print("Employee Name:", self.name)
        print("Employee Salary:", self.salary)

    def give_raise(self, amount):
        self.salary += amount
        print("New Salary after raise:", self.salary)

    def yearly_bonus(self):
        return self.salary * 0.10

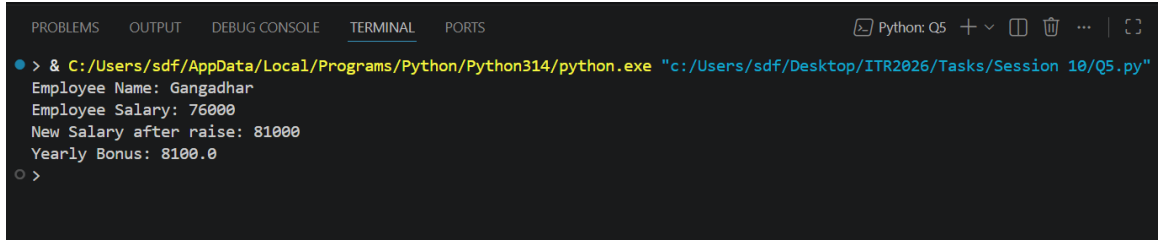
# Create an employee object
emp1 = Employee("Gangadhar", 76000)

# Demonstrate all methods
emp1.display_details()

emp1.give_raise(5000)

bonus = emp1.yearly_bonus()
print("Yearly Bonus:", bonus)
```


Output Screenshot:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the output of a Python script executed from the command line. The output shows the employee name, salary, and the result of a raise calculation.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: Q5 + - [ ] [ ] ... [ ]
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q5.py"
Employee Name: Gangadhar
Employee Salary: 76000
New Salary after raise: 81000
Yearly Bonus: 8100.0
>
```

Q6

Q6. Real-world Object Modeling)

Create a class MobilePhone with attributes: brand, model, battery_percentage.

Add these methods:

charge(percent) — increase battery.

use_phone(minutes) — reduce battery by 1% per 10 minutes.

show_status() — print current status.

Create an object and simulate some usage.

Program Code:

```
# Create a class MobilePhone with attributes: brand, model, battery_percentage.
```

```
class MobilePhone:
```

```
    def __init__(self, brand, model, battery_percentage):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.battery_percentage = battery_percentage
```

```
# Increase battery
```

```
def charge(self, percent):
```

```
    self.battery_percentage += percent
```

```
    if self.battery_percentage > 100:
```

```
        self.battery_percentage = 100
```

```
    print("Battery Percentage:", self.battery_percentage, "%")
```

```
# Reduce battery by 1% per 10 minutes
```

```
def use_phone(self, minutes):
```

```
    battery_used = minutes // 10
```

```
    self.battery_percentage -= battery_used
```

```
    if self.battery_percentage < 0:
```

```
        self.battery_percentage = 0
```

```
    print("Battery after usage:", self.battery_percentage, "%")
```

```
# Show phone status
```

```
def show_status(self):
```

```
    print("\nBrand:", self.brand)
```

```
print("Model:", self.model)
print("Battery:", self.battery_percentage, "%")
```

```
# Create object
phone = MobilePhone("Samsung", "Galaxy S24", 80)
```

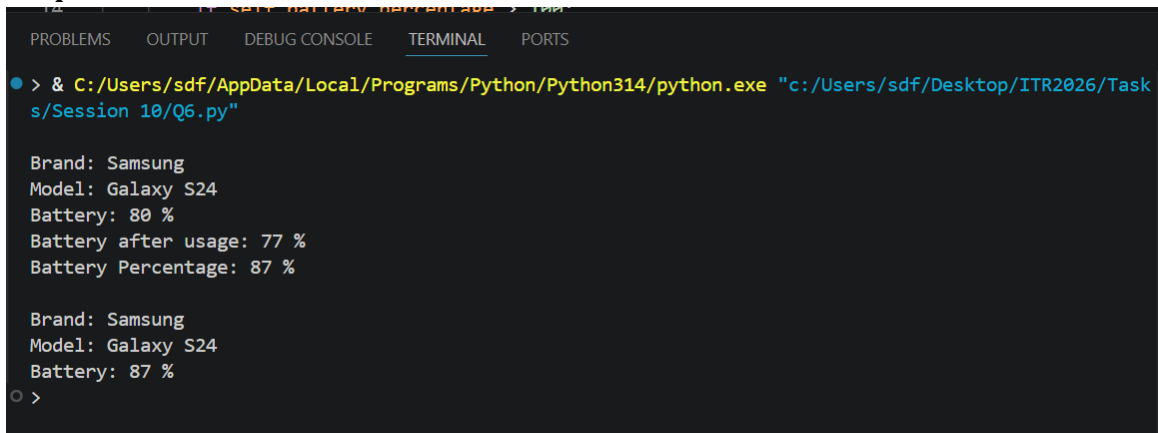
```
# Simulate usage
phone.show_status()
```

```
phone.use_phone(30)  # Battery decreases by 3%
```

```
phone.charge(10)    # Battery increases by 10%
```

```
phone.show_status()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q6.py"

Brand: Samsung
Model: Galaxy S24
Battery: 80 %
Battery after usage: 77 %
Battery Percentage: 87 %

Brand: Samsung
Model: Galaxy S24
Battery: 87 %
>
```

Q7

Q7. Combining Concepts)

Create a class Rectangle with attributes length and width.

Add methods:

area() — return the area.

perimeter() — return the perimeter.

display() — print length, width, area, and perimeter.

Take input from user for length and width, create object, and display results.

Program Code:

```
# Create a class Rectangle with attributes length and width.
```

```
class Rectangle:
```

```
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

```
# area() — return the area.
```

```
    def area(self):
        return self.length * self.width
```

```
# perimeter() — return the perimeter.
```

```
    def perimeter(self):
        return 2 * (self.length + self.width)
```

```
# display() — print length, width, area, and perimeter.
```

```
    def display(self):
        print("\nLength:", self.length)
        print("Width:", self.width)
        print("Area:", self.area())
        print("Perimeter:", self.perimeter())
```

```
# Take input from user
```

```
length = float(input("Enter Length: "))
```

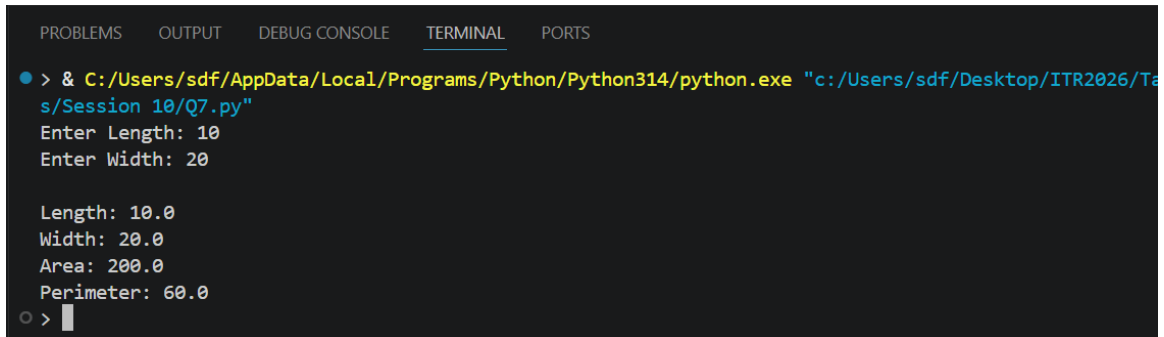
```
width = float(input("Enter Width: "))
```

```
# Create object
```

```
rect = Rectangle(length, width)
```

```
# Display results  
rect.display()
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Ta  
s/Session 10/Q7.py"  
Enter Length: 10  
Enter Width: 20  
  
Length: 10.0  
Width: 20.0  
Area: 200.0  
Perimeter: 60.0  
○ > █
```

Q8

Q8. Updating Attributes)

Create a class Player with attributes name, score, and level.

Add methods:

increase_score(points)

level_up() — increase level by 1.

Session 10 (AIML) - Assignment Questions 2

show_progress()

Create an object, update score and level multiple times, and show progress.

Program Code:

```
class Player:
```

```
    def __init__(self, name, score, level):
```

```
        self.name = name
```

```
        self.score = score
```

```
        self.level = level
```

```
    def increase_score(self, points):
```

```
        self.score += points
```

```
        print(points, "points added.")
```

```
    def level_up(self):
```

```
        self.level += 1
```

```
        print("Level Up!")
```

```
    def show_progress(self):
```

```
        print("\nPlayer Name:", self.name)
```

```
        print("Score:", self.score)
```

```
        print("Level:", self.level)
```

```
# Create object
```

```
player1 = Player("Vaibhav", 100, 1)
```

```
# Show initial progress
```

```
player1.show_progress()
```

```
# Update score and level multiple times
```

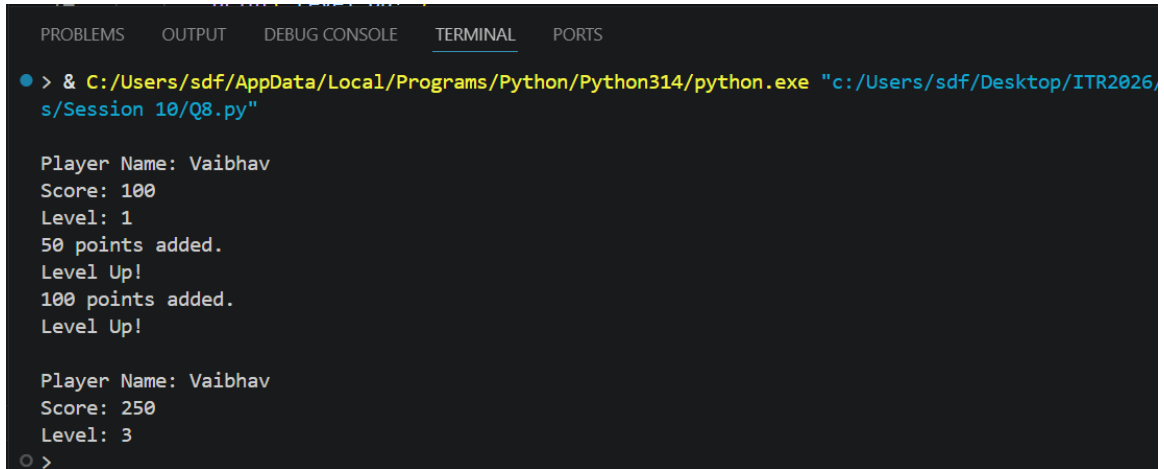
```
player1.increase_score(50)
```

```
player1.level_up()
```

```
player1.increase_score(100)
player1.level_up()
```

```
# Show final progress
player1.show_progress()
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/s/Session 10/Q8.py"

Player Name: Vaibhav
Score: 100
Level: 1
50 points added.
Level Up!
100 points added.
Level Up!

Player Name: Vaibhav
Score: 250
Level: 3
○ >
```

Q9

Q9. Debugging OOP Code)

The following code has errors. Correct it and add proper comments explaining the fixes:

Python

```
class Person:
def __init__(name, age):
name = name
age = age
def introduce():
print("My name is" name "and I am" age "years old.")
p1 = Person("Rahul", 25)
p1.introduce()
```

Program Code:

Create a class Person

```
class Person:
```

```
    # Fixed: Added self parameter
```

```
    def __init__(self, name, age):
```

```
        # Fixed: Store values in object attributes
```

```
        self.name = name
```

```
        self.age = age
```

```
    # Fixed: Added self parameter
```

```
    def introduce(self):
```

```
        # Fixed: Correct print syntax and use object attributes
```

```
        print("My name is", self.name, "and I am", self.age, "years old.")
```

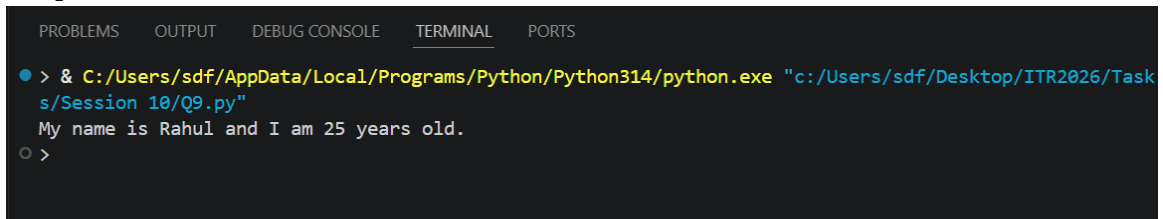
Create object

```
p1 = Person("Rahul", 25)
```

Call method

```
p1.introduce()
```


Output Screenshot:



The screenshot shows a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. Below the tabs, a command is entered: `> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q9.py"`. The output of the command is displayed on the next line: `My name is Rahul and I am 25 years old.`. Below the output, there is a prompt `○ >`.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q9.py"
  My name is Rahul and I am 25 years old.
○ >
```

Q10

Q10. Mini Project - Combined OOP

Create a Simple Library Management System using OOP

Design a class Book with:

Attributes: title, author, status ("Available" or "Issued").

Methods: issue_book(), return_book(), show_info().

Create multiple book objects and demonstrate issuing/returning books with proper messages.

Use a menu (while loop) to allow the user to:

Add a new book

Issue a book

Return a book

Show all books

Session 10 (AIML) - Assignment Questions 3

Exit

Session 10 (AIML) - Assignment Questions 4

Program Code:

```
class Book:
```

```
    def __init__(self, title, author):
```

```
        self.title = title
```

```
        self.author = author
```

```
        self.status = "Available"
```

```
    def issue_book(self):
```

```
        if self.status == "Available":
```

```
            self.status = "Issued"
```

```
            print("Book issued successfully.")
```

```
        else:
```

```
            print("Book is already issued.")
```

```
    def return_book(self):
```

```
        if self.status == "Issued":
```

```
            self.status = "Available"
```

```
            print("Book returned successfully.")
```

```
        else:
```

```
            print("Book is already available.")
```

```
    def show_info(self):
```

```
        print("\nTitle :", self.title)
```

```
print("Author:", self.author)
print("Status:", self.status)
```

```
# List to store book objects
library = []
```

```
while True:
```

```
    print("\n===== Library Management System =====")
    print("1. Add New Book")
    print("2. Issue Book")
    print("3. Return Book")
    print("4. Show All Books")
    print("5. Exit")
```

```
    choice = input("Enter your choice: ")
```

```
    # Add Book
```

```
    if choice == "1":
```

```
        title = input("Enter Book Title: ")
```

```
        author = input("Enter Author Name: ")
```

```
        book = Book(title, author)
```

```
        library.append(book)
```

```
        print("Book added successfully.")
```

```
    # Issue Book
```

```
    elif choice == "2":
```

```
        title = input("Enter Book Title to Issue: ")
```

```
        found = False
```

```
        for book in library:
```

```
            if book.title.lower() == title.lower():
```

```
                book.issue_book()
```

```
                found = True
```

```
                break
```

```
        if not found:
```

```
            print("Book not found.")
```

```
# Return Book
elif choice == "3":
    title = input("Enter Book Title to Return: ")

    found = False

    for book in library:
        if book.title.lower() == title.lower():
            book.return_book()
            found = True
            break

    if not found:
        print("Book not found.")

# Show All Books
elif choice == "4":

    if len(library) == 0:
        print("No books available.")

    else:
        for book in library:
            book.show_info()

# Exit
elif choice == "5":
    print("Thank You!")
    break

else:
    print("Invalid Choice.")
```

Output Screenshot:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

○ > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 10/Q10.py"

===== Library Management System =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter your choice: 1
Enter Book Title: Atomic Habits
Enter Author Name: Jeames Clear
Book added successfully.

===== Library Management System =====
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter your choice: 4

Title : Atomic Habits
Author: Jeames Clear
Status: Available
```